Behaviorbox 2.0

Anonymous authors
Paper under double-blind review

Abstract

What is our abstract?

1 Introduction

The training of large language models (LLMs) is resource-intensive, requiring significant computational costs and development time. Model developers typically track aggregate metrics such as perplexity or accuracy on held-out benchmarks to determine when to stop training or which data to include in different phases across pre-training, mid-training, and post-training. This approach lacks a fine-grained understanding of specific linguistic phenomena, reasoning patterns, or knowledge types that the model learns. Such an understanding could help identify early in training which downstream tasks are likely to succeed or fail, or what training data must be included for success on a set of tasks.

Chen et al. (2025) demonstrate that the model achieving the lowest next-token prediction loss during pre-training does not necessarily perform best after fine-tuning. Instead, probability mass assigned to high-quality outputs better predicts downstream performance. Similarly, Akter et al. (2025) show that reasoning abilities acquired during pre-training may remain dormant until unlocked by fine-tuning. However, these approaches still operate at a coarse-grained level, leaving open questions about which specific dataset features drive these effects. One approach to discover these behaviors involve partitioning evaluation data into slices corresponding to specific behaviors and analyze task performance across these sub-corpora Polyzotis et al. (2019). But these subsets must be known before hand. An alternative approach proposed by Kangaslahti et al. (2025), automatically identifies subsets of data experiencing synchronized changes in loss during training, implying that they rely on the same fine-grained acquired behavior. While interpretable, this method is computationally expensive to scale beyond toy models and datasets.

Our broad goal is to identify behaviors related to fine-grained dataset features across model sizes and tasks that could address the following research questions:

- RQ1: Which features learned during pre-training correlate with performance in a given downstream task?
- RQ2: Are these features really necessary for a given task?

2 Related Work

Age of Acquisition. Some early work on language model training dynamics identifies predictable stages in the acquisition of linguistic abilities. Neural language models first pass through a unigram stage, producing context-agnostic distributions that reflect surface-level frequency statistics (Belrose et al., 2024; Chang et al., 2024). As training progresses, models transition through phases where contextual dependencies and syntactic structures are acquired. A well-studied example is the emergence of induction heads Olsson et al. (2022), Tigges et al. (2024), which marks the onset of in-context learning capabilities. There has also been work comparing linguistic learning curves to human acquisition. Neural models, like children, acquire more frequent words earlier, but unlike children, they learn polysemous short words later and show different part-of-speech acquisition patterns Chang & Bergen (2022). An important direction to study as language models have grown in scale is to compare different sizes of models. Chang et al. (2024) find that tokens can be

categorized as stagnating, forgetting, or learning based on their perplexity slopes, and that small models often remain trapped in low-quality distributions (e.g., grammatical but semantically implausible text), whereas larger models escape these basins. Diehl Martinez et al. (2024) study training under different random seeds and show that larger models reconverge more reliably and that function words converge earliest. A parallel line of work focuses on the evolution of internal mechanisms rather than surface metrics. Tigges et al. (2024) show that circuit-level mechanisms emerge in consistent stages across training and scale, suggesting consistent ordering in how capabilities form.

Discovery of Fine-Grained Behavior. The methods discussed in the previous subsection are limited to testing a few broad hypotheses about learning dynamics. In this section, we discuss some approaches to automatically discovering fine-grained behaviors. Tjuatja & Neubig (2025) introduce BehaviorBox, which compares model outputs across datasets to discover fine-grained regions (e.g., archaic spellings, conversational markers) where one model excels. This method can be easily extended to compare n models or even checkpoints of the same model across training. Kangaslahti et al. (2025) cluster examples for a given task whose loss curves change synchronously across training, revealing subsets that rely on shared underlying capabilities. There are also methods that look at fine-grained features from the perspective of model internal representations. Boháek et al. (2025) identify fine-grained gaps in model performance and benchmark coverage using concepts represented by concept activations from pre-trained SAEs. To track concept activations over time, Bayazit et al. (2025) track the development of linguistic representations using crosscoders trained on checkpoint triplets.

Pre-training Indicators of Downstream Performance. While most improvements in downstream performance are often attributed to scaling model parameters and data, there has now been a significant body of work that shows that there are more fine-grained features of training data composition that could affect downstream performance. From analysing the performance of over 90 language models, Liu et al. (2025) identify the importance of specific data domains for a variety of downstream tasks. There is also work that analyses the impact of a single domain of data such as code on reasoning performance Aryabumi et al. (2024), Petty et al. (2025), or the impact on multilingual performance on increasing the proportion of English data in the pre-training mix Yue et al. (2025). Some studies report further fine-grained indicators beyond data domains such as the presence of parallel structures in texts contributing to in-context learning abilities Chen et al. (2024). In a line of work similar to ours but from the perspective of model internal representations, Chen et al. (2026) show that the distance between representations of different syntactic structures increases throughout training and correlate with increased performance on complex reasoning tasks.

These findings motivate our research: if fine-grained pre-training features have predictable relationships with downstream performance on certain tasks, we need a generalisable method that can find these features for any given task and model.

3 Method

3.1 Overview

This work consists of three main stages as shown in fig 1: (1) data preparation to create performance-aware contextual embeddings, (2) training a Sparse Autoencoder (SAE) to extract features, (3) labeling and comparing features and trajectories.

3.1.1 Model and Checkpoint Selection

3.1.2 Data Preparation

Performance-Aware Contextual Representations Following BehaviorBox Tjuatja & Neubig (2025), we construct a performance-aware contextual representation for each token by concatenating token embeddings from Longformer Beltagy et al. (2020) with language model token probabilities from n checkpoints. This concatenated representation has dimensionality

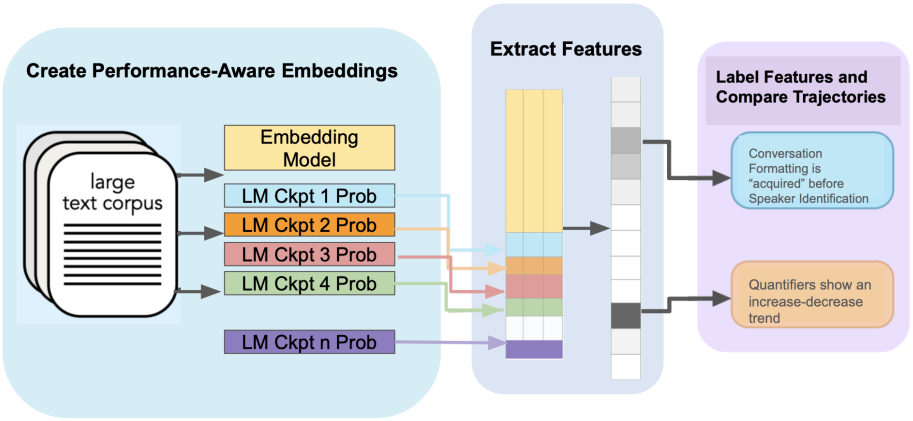


Figure 1: Overview of the proposed .

d_{embed}+$n_{checkpoints}$ where d_{embed} is the model's embedding dimension and $n_{checkpoints}$ is the number of checkpoints.

3.1.3 *Sparse Autoencoder Training*

Exactly as in BehaviorBox, we train a BatchTopK Sparse Autoencoder (SAE) **??** on the performance-aware contextual representations. The general formulation of an SAE involves of an encoder and decoder: the encoder takes as input a vector x, which is a concatenation of the contextual word em- bedding and LM probabilities, and creates a sparse representation $f(x)$ as in equation 1, where $\sigma(.)$ is the activation function and W_{enc} and b_{enc} are the weights of the encoder.

The decoder then reconstructs the input as $\hat{x}$ from this sparse representation as in equation 2, where W_{dec} and b_{dec} are the weights of the decoder.

$$f(x) = \sigma(W_{\text{enc}}x + b_{\text{enc}}) \tag{1}$$

$$\hat{x} = f(x)W_{\text{dec}} + b_{\text{dec}} \tag{2}$$

Fine-grained discovery is enabled by using an overcomplete architecture (far more features than input dimensions) combined with strong sparsity constraints.**?**. This sparsity constraint is enforced through the BatchTopK SAE (equation 3), where for each batch of inputs to the SAE of size N, we flatten the batch and zero out all activations that are not in the top $N \times k$ activations. This encourages each feature to activate on narrow, interpretable patterns and achieve **coherence**.

$$f_{\text{sparse}}(x) = \text{BatchTopK}\big(\sigma(W_{\text{enc}}x + b_{\text{enc}}), k\big) \tag{3}$$

The dimension of the sparse representation we learn is 3000 with sparsity constraint k = 50. **Performance differences** are captured by upweighting the probability scores from the checkpoints to ensure the SAE also learns features that distinguish checkpoint behaviors rather than just semantic similarities without being overwhelmed by the large number of embedding features' contribution. We up-weigh the probability features such that the magnitude of the probability components make up 70% of the total magnitude of the input.

For the activation function, we use RELU Agarap (2018). The weights of the encoder and decoder are learned by minimising the L2 distance between x and $\hat{x}$ using the AdamW optimizer **?**.

3.1.4 *Feature Identification*

We use the same process of labeling and filtering features using LLMs as in BehaviorBox. We add another step of categorizing each feature based on the trend exhibited by the language model probabilities from the earliest to the final checkpoint.

3.1.5 *Trend Classification Method*

We classify each feature's learning curve of median checkpoint probabilities using a set of intuitive rules. We first smooth the learning curve $x_1, x_2, ...x_n$ by applying a moving average with window size 3 to reduce local noise:

$$\tilde{x}_i = \begin{cases} x_1, & i = 1, \\ \frac{x_{i-1}+x_i+x_{i+1}}{3}, & 2 \leq i \leq n-1, \\ x_n, & i = n. \end{cases} \tag{4}$$

We then normalize the smoothed curve to lie between 0 and 1:

$$y_i = \frac{\tilde{x}_i - \min_j \tilde{x}_j}{\max_j \tilde{x}_j - \min_j \tilde{x}_j} \tag{5}$$

We then extract a the following set of basic properties from the normalized values:

- **Start/End Values:** taken directly from the first y_1 and last points y_n.
- **Peak/Trough:** identified by the maximum $\max_i y_i$ or minimum $\min_i y_i$ point along the curve.
- **Overall Change:** difference between end and start values $y_n - y_1$.
- **Upward/Downward Steps:** the number of positive, negative, and zero (difference less than 0.01) steps calculated based on first-order differences $y_{i+1} - y_i$ across checkpoints.
- **Final Stagnation:** the standard deviation (std) of the last 30% of the curve, indicating whether learning has plateaued (if the std is smaller than 0.08).

These signals allow us to classify trajectories into intuitive categories such as

- *increase-decrease*: If the rise and fall are atleast 0.3, where rise is the difference between the peak and start value, and fall is the difference between the peak and end value.
- *decrease-increase*: If the rise and fall are atleast 0.3, where rise is the difference between the end value and trough, and fall is the difference between the start value and trough.
- *increasing*: If the overall change is atleast 0.2, the number of positive steps is more than the number of negative steps, and there is no final stagnation.
- *decreasing*: If the overall change is atleast 0.2, the number of positive steps is less than the number of negative steps, and there is no final stagnation.
- *increase-stagnate* or *decrease-stagnate*: If final stagnation is seen after the first two tests of increasing or decreasing trends respectively.
- *other*: Any pattern that does not fall into the above categories

This classification helps identify continuous learning or learned (increase patterns), and forgetting (decrease patterns).

These thresholds (e.g. for stagnation, rise, fall) have not been varied and are a bit arbitrary as this is a preliminary analysis report.

4 Experiments

4.1 RQ1:Which features learned during pre-training correlate with performance in a given downstream task?

4.1.1 Correlation

There could be features that correlate strongly with downstream performance across tasks and others that correlate strongly only with certain tasks.

Pearson correlation: measures the linear relationship between two variables.

Spearman correlation: measures the rank order relationship. It is more robust to outliers and does not assume a linear relationship.

When features are composed of predominantly of perfectly monotonically increasing/decreasing probabilities, the spearman correlation will be very similar to the pearson correlation score. Additionally, since performance tends to usually increase with training for most tasks, and if most tokens see an increase in probability with training, it can create spurious correlations caused by training progress instead of by any meaningful relationship between the feature and the task.

To mitigate these spurious correlations, we compute residual correlations by measuring the relationship between feature probabilities x and task performance y while controlling for a baseline variable z, which is the median probability across all tokens at each training step.

To remove the shared variance due to general improvement in language model training, we first regress out z from both x and y separately as follows:

$$x_i = \alpha_x + \beta_x z_i + \epsilon_{x,i}, \tag{6}$$
$$y_i = \alpha_y + \beta_y z_i + \epsilon_{y,i}, \tag{7}$$

where $\epsilon_{x,i}$ and $\epsilon_{y,i}$ are the residuals that cannot be explained by relationship with the baseline median probability.

The residual correlation is thus the correlation between the two residuals:

$$\rho_{xy \cdot z} = \text{corr}(\epsilon_x, \epsilon_y) \tag{8}$$

4.1.2 Common Features Across Models

For every task, we look at features with high residual pearson correlation to obtain relevant features. To identify if there are common features identified across models, we compute a weighted combination of embedding-based similarity ($\langle e_i, e_j \rangle$) and TF-IDF vector similarity ($\langle t_i, t_j \rangle$) to find the best match for a feature from model i with that of model j:

$$\text{Match}(i) = \arg\max_j \left[(1 - \lambda) \langle e_i, e_j \rangle + \lambda \langle t_i, t_j \rangle \right]$$

We set λ to 0.4

4.2 RQ2:Are these features really necessary for a given task?

To evaluate this, we obtain tokens corresponding to highly positively correlated features ($\geq$0.8), highly negatively correlated features ($\leq$-0.8), and neutral features ([-0.2,0.2]).

Ideally, training on tokens from positively correlated features should boost performance on the task and training on negatively correlated features should reduce performance on a task. To test this, we create training tokens corresponding to positive + neutral features and negative + neutral features.

We hope that the continuing to train the final checkpoint in our analysis on both these training token sets should improve performance in one case and reduce in the other.

Since these tokens are a part of documents, we use a masking strategy to include the desired tokens only in the loss computation.

5 Results

5.1 RQ1

5.1.1 Correlated Features per Task

How to show this

5.1.2 Common Features

We find 97 common features between Amber and Olmo with the best match similarity of at least 0.8.

Table 1: Grouped Common Features with Examples

Category	**Examples**
Formatting & Whitespace	Tab indentation (\t), mixed spaces/tabs, alignment spacing, paragraph separators
Punctuation	Commas (,), periods (.), quotation marks (”), exclamation marks (!)
Numbers	1,000 (thousands separator), years (2023), citation indices ([12]), large values (“millions”)
Grammar (Function Words, Pronouns, Modals)	Articles: “the”; Pronouns: “I”, “you”, “this”; Modals: “can”, “could”, “will”; Prepositions: “to”, “on”, “with”
Clause / Linking Words	and, or, but, subordinators such as although, because
Discourse Markers & Idioms	having said that, for the time being, state-of-the-art, informal tags like right?, you know?
Code-Specific Tokens	Assignment operators (=), variable names (data_frame), architecture terms (64-bit)
Academic / Citation Signals	Author initials (J. K. Rowling), citation years (Smith, 2020), numbered sections
Domain-Specific Vocabulary	Healthcare: clinical care, healthcare system; Geology: mineral deposits, gold reserves; Anatomy: upper torso; Presentations: keynote speech

5.1.3 Median correlation versus task performance

To see if there is a relationship between the strength of correlation and the average task performance for a model, we plot figure 2.

5.2 RQ2

For Amber - Classified training tokens for each task and plotted proportion of tokens for each task in figure 3

Acknowledgments

Use unnumbered first level headings for the acknowledgments. All acknowledgments, including those to funding agencies, go at the end of the paper.

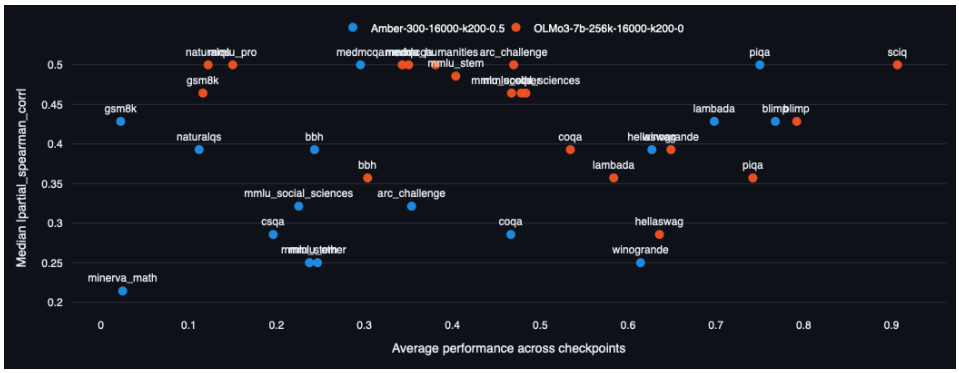


Figure 2: Avg performance vs median |correlation|

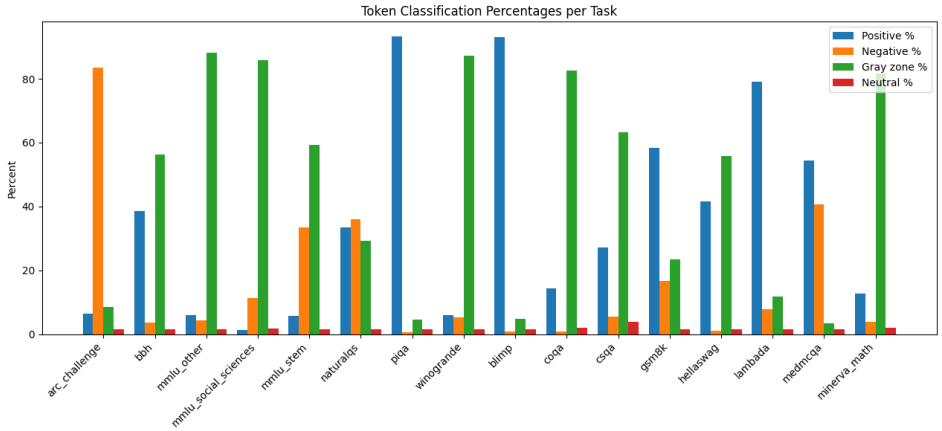


Figure 3: Amber Token Classes Per Task

Ethics Statement

Authors can add an optional ethics statement to the paper. For papers that touch on ethical issues, this section will be evaluated as part of the review process. The ethics statement should come at the end of the paper. It does not count toward the page limit, but should not be more than 1 page.

References

Abien Fred Agarap. Deep learning using rectified linear units (relu). *ArXiv*, abs/1803.08375, 2018. URL https://api.semanticscholar.org/CorpusID:4090379.

Syeda Nahida Akter, Shrimai Prabhumoye, Eric Nyberg, Mostofa Patwary, Mohammad Shoeybi, Yejin Choi, and Bryan Catanzaro. Front-loading reasoning: The synergy between pretraining and post-training data, 2025. URL https://arxiv.org/abs/2510.03264.

Viraat Aryabumi, Yixuan Su, Raymond Ma, Adrien Morisot, Ivan Zhang, Acyr Locatelli, Marzieh Fadaee, Ahmet Üstün, and Sara Hooker. To code, or not to code? exploring impact of code in pre-training, 2024. URL https://arxiv.org/abs/2408.10914.

Deniz Bayazit, Aaron Mueller, and Antoine Bosselut. Crosscoding through time: Tracking emergence consolidation of linguistic representations throughout llm pretraining, 2025. URL https://arxiv.org/abs/2509.05291.

Nora Belrose, Quintin Pope, Lucia Quirke, Alex Mallen, and Xiaoli Fern. Neural networks learn statistics of increasing complexity. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Matyá Boháek, Nino Scherrer, Nicholas Dufour, Thomas Leung, Christoph Bregler, and Stephanie C. Y. Chan. Uncovering competency gaps in large language models and their benchmarks. *ArXiv*, abs/2512.20638, 2025. URL https://api.semanticscholar.org/CorpusID:284154152.

Tyler A. Chang and Benjamin K. Bergen. Word acquisition in neural language models. *Transactions of the Association for Computational Linguistics*, 10:1–16, 2022. doi: 10.1162/tacl_a_00444. URL https://aclanthology.org/2022.tacl-1.1/.

Tyler A. Chang, Zhuowen Tu, and Benjamin K. Bergen. Characterizing learning curves during language model pre-training: Learning, forgetting, and stability. *Transactions of the Association for Computational Linguistics*, 12:1346–1362, 2024. doi: 10.1162/tacl_a_00708. URL https://aclanthology.org/2024.tacl-1.74/.

Fan Chen, Audrey Huang, Noah Golowich, Sadhika Malladi, Adam Block, Jordan T. Ash, Akshay Krishnamurthy, and Dylan J. Foster. The coverage principle: How pre-training enables post-training, 2025. URL https://arxiv.org/abs/2510.15020.

Michelle Chao Chen, Moritz Miller, Bernhard Schölkopf, and Siyuan Guo. On the emergence and test-time use of structural information in large language models, 2026. URL https://arxiv.org/abs/2601.17869.

Yanda Chen, Chen Zhao, Zhou Yu, Kathleen McKeown, and He He. Parallel structures in pre-training data yield in-context learning, 2024. URL https://arxiv.org/abs/2402.12530.

Richard Diehl Martinez, Pietro Lesci, and Paula Buttery. Tending towards stability: Convergence challenges in small language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 3275–3286, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.187. URL https://aclanthology.org/2024.findings-emnlp.187/.

Sara Kangaslahti, Elan Rosenfeld, and Naomi Saphra. Hidden breakthroughs in language model training. *arXiv preprint arXiv:2506.15872*, 2025.

Emmy Liu, Amanda Bertsch, Lintang Sutawika, Lindia Tjuatja, Patrick Fernandes, Lara Marinov, Michael Chen, Shreya Singhal, Carolin Lawrence, Aditi Raghunathan, Kiril Gashteovski, and Graham Neubig. Not-just-scaling laws: Towards a better understanding of the downstream impact of language model design decisions. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 16396–16427, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.830. URL https://aclanthology.org/2025.emnlp-main.830/.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads. *Transformer Circuits Thread*, 2022. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

Jackson Petty, Sjoerd van Steenkiste, and Tal Linzen. How does code pretraining affect language model task performance?, 2025. URL https://arxiv.org/abs/2409.04556.

Dataset	Tokens	Hidden Dim	Top-K
Olmo3	6148021	3000	50

Table 2: Training tokens for SAE

Neoklis Polyzotis, Steven Whang, Tim Klas Kraska, and Yeounoh Chung. Slice finder: Automated data slicing for model validation. In *Proceedings of the IEEE Int' Conf. on Data Engineering (ICDE), 2019*, 2019. URL https://arxiv.org/pdf/1807.06068.pdf.

Curt Tigges, Oskar J. Hollinsworth, Atticus Geiger, and Neel Nanda. Language models linearly represent sentiment. In Yonatan Belinkov, Najoung Kim, Jaap Jumelet, Hosein Mohebbi, Aaron Mueller, and Hanjie Chen (eds.), *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pp. 58–87, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.5. URL https://aclanthology.org/2024.blackboxnlp-1.5/.

Lindia Tjuatja and Graham Neubig. BehaviorBox: Automated discovery of fine-grained performance differences between language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 18851–18873, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.923. URL https://aclanthology.org/2025.acl-long.923/.

Xiang Yue, Yueqi Song, Akari Asai, Seungone Kim, Jean de Dieu Nyandwi, Simran Khanuja, Anjali Kantharuban, Lintang Sutawika, Sathyanarayanan Ramamoorthy, and Graham Neubig. Pangea: A fully open multilingual multimodal llm for 39 languages, 2025. URL https://arxiv.org/abs/2410.16153.

A Appendix

You may include other additional sections here.

A.1 Model Details

We use the following models and their checkpoints for our analysis:

1. Olmo 3 Base 7B [1] - stage1-step1000, stage1-step15000, stage1-step73000, stage1-step146000, stage1-step365000, stage1-step731000, stage1-step1096000, stage1-step1169000, stage1-step1315000, stage2-step18000, main

A.2 Dataset Details

Olmo3 validation corpus. [2] This corpus contains 1626 documents from various domains described in figure:

A.3 SAE Training Details

A.4 Evaluation Tasks

A.4.1 Olmo3-7b

[1] https://huggingface.co/allenai/Olmo-3-1025-7B
[2] Indices for validation documents

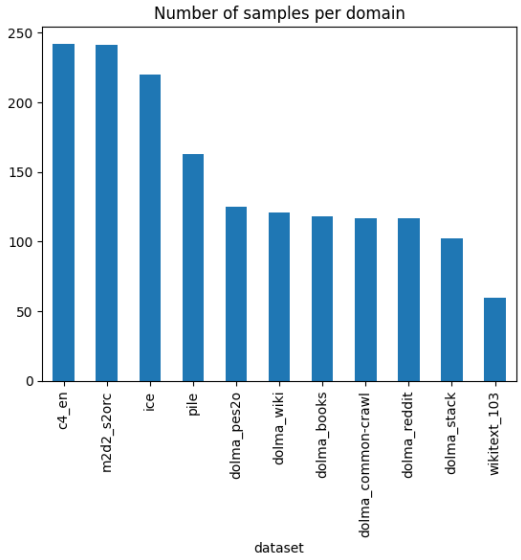
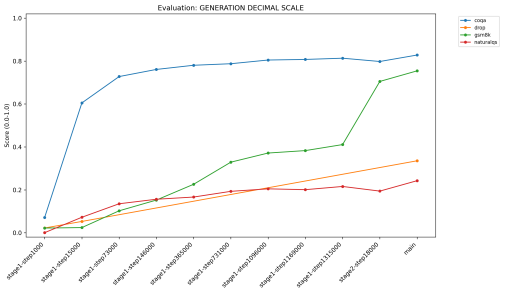
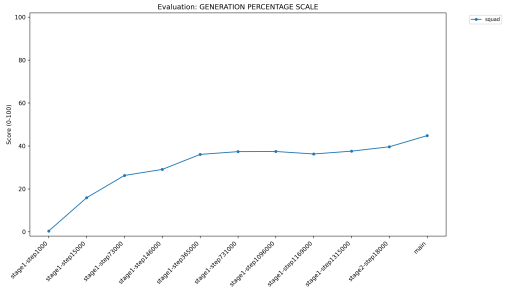


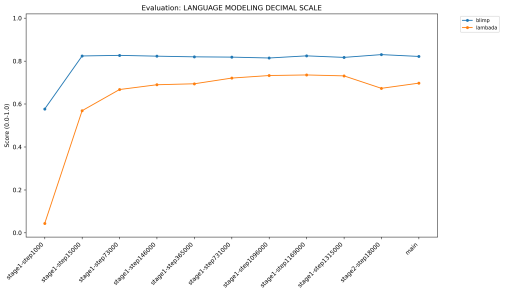
Figure 4: Sample figure caption.



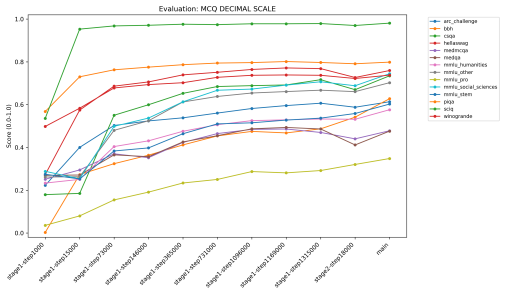
(a) Generation (Decimal)



(b) Generation (Percentage)



(c) Language Modeling (Decimal)



(d) MCQ (Decimal)

Figure 5: Evaluation metrics for OLMo3-7B across different scales and tasks.